

Flowvelly Healthcare Production Readiness Review & Runbook

Flowvelly Healthcare Production Readiness Review + Runbook

Context and assumptions

- Target workload: production GKE inference service for Flowvelly Healthcare, backed by Cloud SQL Postgres and Cloud Storage, operating in a Shared VPC host project.
- Host-project networking is managed by a partner team; service team owns the GKE deployment, probes, app-level Cloud SQL connectivity, and workload identity.
- Environment has QA / PreProd / Prod folders in a GCP org and must support audit evidence, least privilege, and PHI-bordering restrictions.
- Observability is currently light; assume Monitoring/Logging exist but lack mature SLIs/SLOs and PHI-safe structured logging.
- Assume CI/CD mix may include GitHub Actions or GitLab, with production promotion gating not fully consistent across teams.

Readiness review

Summary table

- Reliability: needs-work
- Bursty / cost-sensitive scaling: needs-work
- Least privilege: needs-work
- Observability / SLIs-SLOs: blocker
- Data handling / PHI safety: needs-work
- Change management / audit evidence: needs-work
- Ownership & handoff: needs-work

1. Reliability

- Status: needs-work
- Reason: service is already exposed to intermittent 5xx with normal latency on successful requests, suggesting a subset failure domain rather than a full-service outage.
- Findings: partner-managed Shared VPC networking changes are a single point of failure; current operational checks do not clearly distinguish unhealthy backends, partial routing issues, or Cloud SQL connection failures.
- Required improvements:
 - add explicit LB/NEG health monitoring and alerting for unhealthy backend endpoints.
 - add a production runbook for partial backend failure and host-network change correlation.
 - verify health checks and readiness probes reflect actual inference success conditions.

2. Bursty / cost-sensitive scaling

- Status: needs-work

- Reason: the workload likely sees bursty inference demand, but there is no documented autoscaling or cost trim strategy.
- Required improvements:
 - define GKE HPA policies on request concurrency or CPU and cap maximum node counts.
 - pair Cloud SQL sizing and connection pooling with burst capacity.
 - consider pre-warmed node pools or minimum replicas in prod for latency-sensitive inference.

3. Least privilege

- Status: needs-work
- Reason: least privilege is assumed but not explicitly validated for GKE, Cloud SQL, Cloud Storage, and Shared VPC boundaries.
- Required improvements:
 - enforce Workload Identity for GKE service accounts and avoid static DB credentials in code or Terraform state.
 - restrict Cloud Storage bucket access to only required service accounts, not broad project viewers.
 - ensure partner host-project roles do not overreach into service project data plane and that service accounts only have narrow IAM roles.

4. Observability with real SLIs/SLOs

- Status: blocker
- Reason: there are no concrete, documented SLOs; current instrumentation is generic and insufficient for production gatekeeping.
- Required improvements:
 - define at least two SLIs: prod inference request success rate and end-to-end latency.
 - set an SLO such as 99.9% success for inference requests over a 30-day window and 95th percentile latency below an agreed threshold (example: 500ms for inference, subject to business need).
 - build dashboards for LB 5xx ratio, NEG endpoint health, GKE readiness probe failures, Cloud SQL failed connections, and request distribution.

5. Data handling and PHI safety

- Status: needs-work
- Reason: PHI-bordering data requires explicit logging and audit controls; current documentation lacks redaction and retention policy details.
- Required improvements:
 - use structured, metadata-only logging for request IDs, error categories, subsystem, and status codes; do not capture payloads or PHI-related headers.
 - enable Cloud Audit Logs for admin and data access, and retain them according to healthcare compliance requirements.
 - enforce CMEK or default encryption on Cloud Storage, and lock down bucket ACLs and object-level permissions.

6. Change management and audit evidence

- Status: needs-work
- Reason: there is a known shared-host Terraform change, but production change control and audit artifact capture appear immature.
- Required improvements:
 - standardize a promotion workflow with approvals for `preprod` and `prod`, including explicit evidence of who approved and what was changed.
 - preserve Terraform plan/app diffs, merge approvals, and change request IDs for network and application changes.
 - require Shared VPC host-project change reviews prior to apply, especially for firewall, route, and NAT modifications.

7. Ownership and escalation

- Status: needs-work
- Reason: the cross-team ownership model is recognized, but there is no formal escalation path or documented boundary in the runbook.
- Required improvements:
 - document partner vs app team ownership: partner owns Shared VPC host project networking, app team owns GKE deployment and Cloud SQL schema.
 - publish on-call escalation contacts and verify they have appropriate access.
 - ensure runbooks reference the correct team handoff for production and compliance evidence.

Production runbook: intermittent prod 5xx on inference service

Failure signature

- Symptom: ongoing intermittent 5xx responses from the prod inference service, approximately 1 in 6 requests.
- Good requests have normal latency.
- Known context: no app deploy today; partner merged a host-project network Terraform change this morning; Cloud SQL CPU is normal; PHI-bordering environment forbids verbose payload logging.

Primary hypothesis

- A subset of backend endpoints or Shared VPC path is failing, most likely due to a host-project networking change affecting specific zones, node pools, or NAT/route paths.

First checks

1. Confirm impact, scope, and timeline

- Verify whether the issue is prod only and whether the failure rate is sustained or increasing.

- Use Monitoring/LB logs to confirm the error window and the percentage of 5xx traffic.

- Example MQL for LB 5xx ratio:

```
...
```

```
fetch gce_http_load_balancer_request_count
| metric 'loadbalancing.googleapis.com/https/request_count'
| filter (response_code >= 500 && response_code < 600)
| group_by 1m, [value_request_count_sum]
| ratio
```

```
...
```

- Correlate with the partner network change timestamp from Cloud Audit Logs.

2. Check prod LB backend health

- Run:

```
```bash
```

```
gcloud compute backend-services get-health PROD-BACKEND-SERVICE \
--project=PROD_HOST_PROJECT --global
```

```
...
```

- For regional backends, add `--region=REGION`.

- Look for any backend showing `UNHEALTHY`, `DRAINING`, or status differences across zones.

- Note whether the failure domain is one backend service, one zone, or one NEG.

## ***3. Inspect GKE deployment and pod health***

- Run:

```
```bash
```

```
kubectl get pods -n PROD_NAMESPACE -l app=inference -o wide
kubectl describe pod $(kubectl get pods -n PROD_NAMESPACE -l app=inference -o
jsonpath='{.items[0].metadata.name}') -n PROD_NAMESPACE
kubectl get events -n PROD_NAMESPACE --sort-by=.lastTimestamp | tail -50
```

```
...
```

- Confirm readiness/liveness probe failures, OOMKills, image pull errors, or node affinity failures.

- Determine whether only a subset of pods or nodes are failing.

4. Assess Cloud SQL connectivity

- In Monitoring, inspect Cloud SQL connection metrics:

- `cloudsql.googleapis.com/database/failed\_connections`

- `cloudsql.googleapis.com/database/connection\_count`

- `cloudsql.googleapis.com/database/cpu/utilization`

- If you can query the database:

```
```sql
```

```
SELECT state, count(*) FROM pg_stat_activity GROUP BY state;
SELECT count(*) FROM pg_stat_activity WHERE wait_event IS NOT NULL;
...
```

- Check whether failed connections spike in the same window as 5xx errors.

## **5. Validate Shared VPC host network change**

- Identify the exact Terraform change and its apply timestamp.
- Query Cloud Audit Logs for the host project to find relevant network operations:
  - ``protoPayload.methodName="beta.compute.firewalls.patch"```
  - ``protoPayload.methodName="beta.compute.routes.patch"```
  - ``protoPayload.methodName="beta.compute.forwardingRules.patch"```
- If VPC Flow Logs are enabled, look for denied or dropped traffic to Cloud SQL and GKE node IPs.

## **Decision points**

### ***Decision: host-network change confirmed***

- If backend health is degraded in specific zones and the timing matches the host-project change, classify this as a host-network regression.
- Mitigation: roll back the network Terraform change immediately and validate LB backend health.
- Escalate to partner network team and preserve the change diff for audit.

### ***Decision: GKE/NODE subset failure***

- If a subset of pods or nodes are failing with readiness/liveness issues, treat this as an app or node-level problem.
- Mitigation: cordon/drain unhealthy nodes, restart failing pods, and inspect recent app config changes.
- Confirm Cloud SQL connectivity and avoid broad restarts without addressing the underlying failure.

### ***Decision: Cloud SQL connectivity issue***

- If Cloud SQL failed connections or proxy resets are the dominant signal, escalate to the app team and verify Cloud SQL private IP/firewall path.
- Mitigation: if connection saturation is found, reduce app connection pool sizes or scale Cloud SQL read replicas / instance sizing as needed.

## **Mitigations**

- Host-network rollback:
  - restore the last known-good Shared VPC firewall/route/NAT config.
  - verify ``gcloud compute backend-services get-health`` shows all backends healthy.
- Partial backend isolation:

- temporarily remove unhealthy backends from the LB with `gcloud compute backend-services remove-backend` or by draining the NEG.
- App recovery:
- restart failing inference pods after confirming the underlying cause is not networking.
- avoid enabling verbose logging; instead use structured error metadata and request IDs.

## Escalation path

- Partner host-team: Shared VPC, firewall, route, Cloud NAT.
- App team: GKE deployment, readiness/liveness probes, Cloud SQL proxy/config.
- If the issue affects PHI data handling or audit evidence, escalate to compliance/Platform governance.
- Document all decisions and approvals in the incident channel and retain a timeline for audit.

## Rollback guidance

- Prefer rollback of the host-project network change if that is the confirmed root cause.
- If the app deployment is responsible and there was a recent production promotion, use the CI/CD rollback pipeline with documented approval.
- In all cases, preserve the rollback rationale, timestamp, operator identity, and the exact diff in the incident record.

## Alert definitions to add

- Prod LB 5xx ratio > 10% for 5 minutes.
- Prod NEG unhealthy endpoint count > 0.
- Cloud SQL failed connection count > 5 over 5 minutes.
- GKE readiness probe failures > 10% across inference pods in prod.

## AI judgment and critical corrections

- I used AI as a first draft source, then validated it against the healthcare incident context.
- I corrected the AI's generic behavior by adding a concrete 1-in-6 failure-domain interpretation and by avoiding generic "check the logs" guidance.
- I added healthcare-specific items: PHI-safe structured logging, Cloud Audit Log change correlation, Shared VPC partner/app ownership, and audit-ready rollback evidence.
- I also included a blocker-level finding for missing SLIs/SLOs, since production readiness depends on measurable service-level objectives rather than checklists.

## AI Companion Note

- I used an AI assistant to draft the incident review and runbook, then reviewed it against the Flowvelly Healthcare scenario.
- Corrected two concrete shortfalls: the missing subset-failure hypothesis for ~16% 5xx and the PHI-safe logging requirement that forbids payload capture.
- Added details the AI did not supply: explicit LB/NEG health commands, Cloud Audit Log query targets, team ownership boundaries, and SLI/SLO rationale.

- I kept the output grounded by labeling missing evidence as blocker items and by documenting required changes rather than assuming the environment is already compliant.